STORAGE



Direct Write QLC: The End of SLC Caching

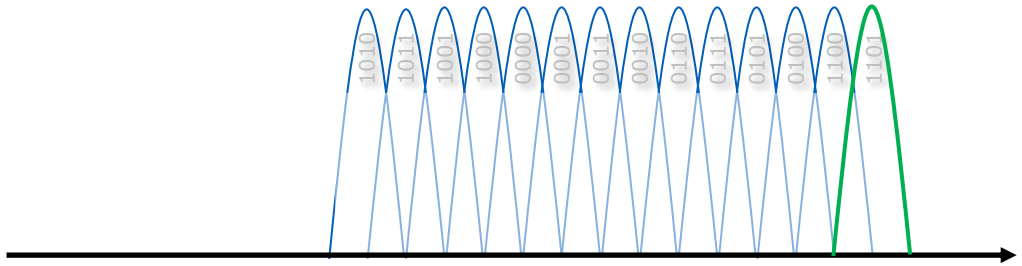
Stephen Gold

Distinguished Engineer, SANDISK

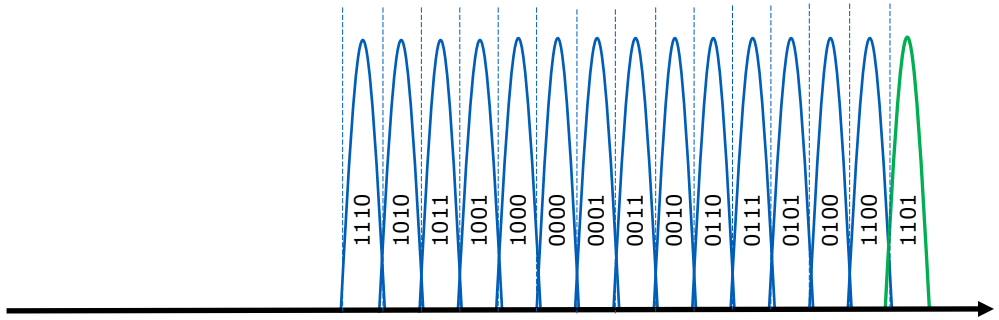
OCP BRONZE™ Member

Why is QLC more difficult than TLC?

- To read data from the cell, you need to be able to differentiate between the different voltage states
- QLC has 4 bits per cell, which means there are 16 possible data states the cell can be in, vs 8 data states for the TLC
- TLC and QLC share the similar Vt window, however TLC needs to squeeze 8 states into that voltage window, QLC needs to squeeze in 16 states. This means the read margin (state to state separation) is different
- If the states overlap too much after a program, then the data is not readable



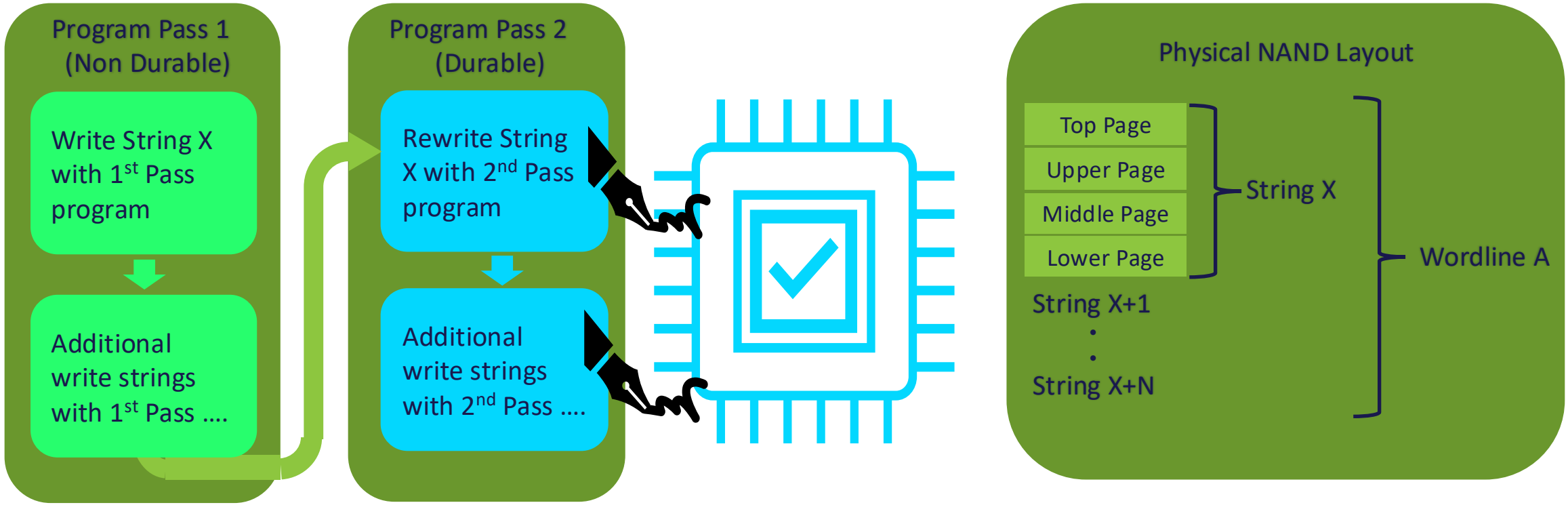
States overlapping too much
NOT Readable



State overlaps within ECC limits
Readable

How does QLC traditionally write data?

QLC writes data in two programming passes to ensure durability



Two-Pass Programming vs Scaling of NAND (Capacity/Performance)

First Pass Programming **is not** Durable

≠

Enterprise SSDs Guarantee Durability even with power loss

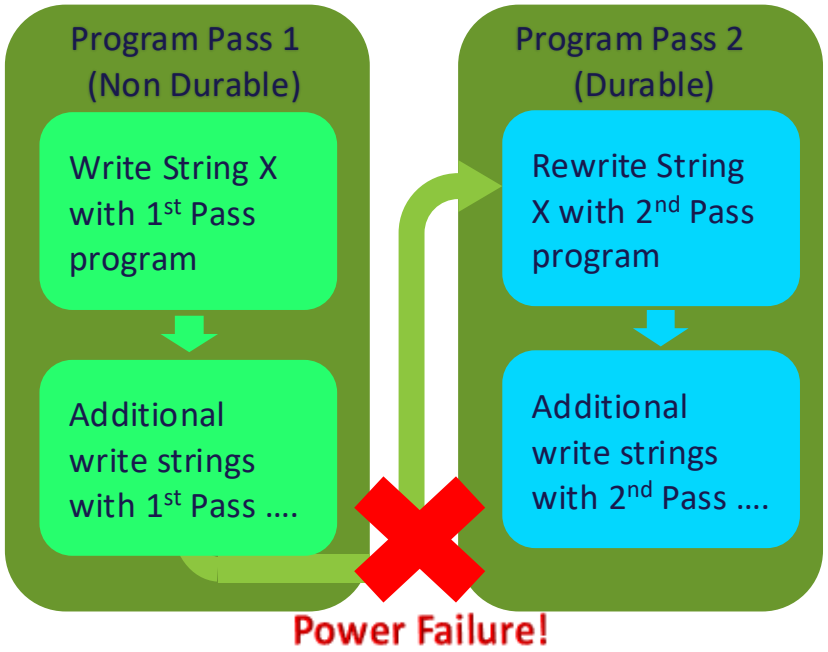
What Problems Does This Present?

First pass data will at times be measured in the **100s of Mbytes**

Data at Risk

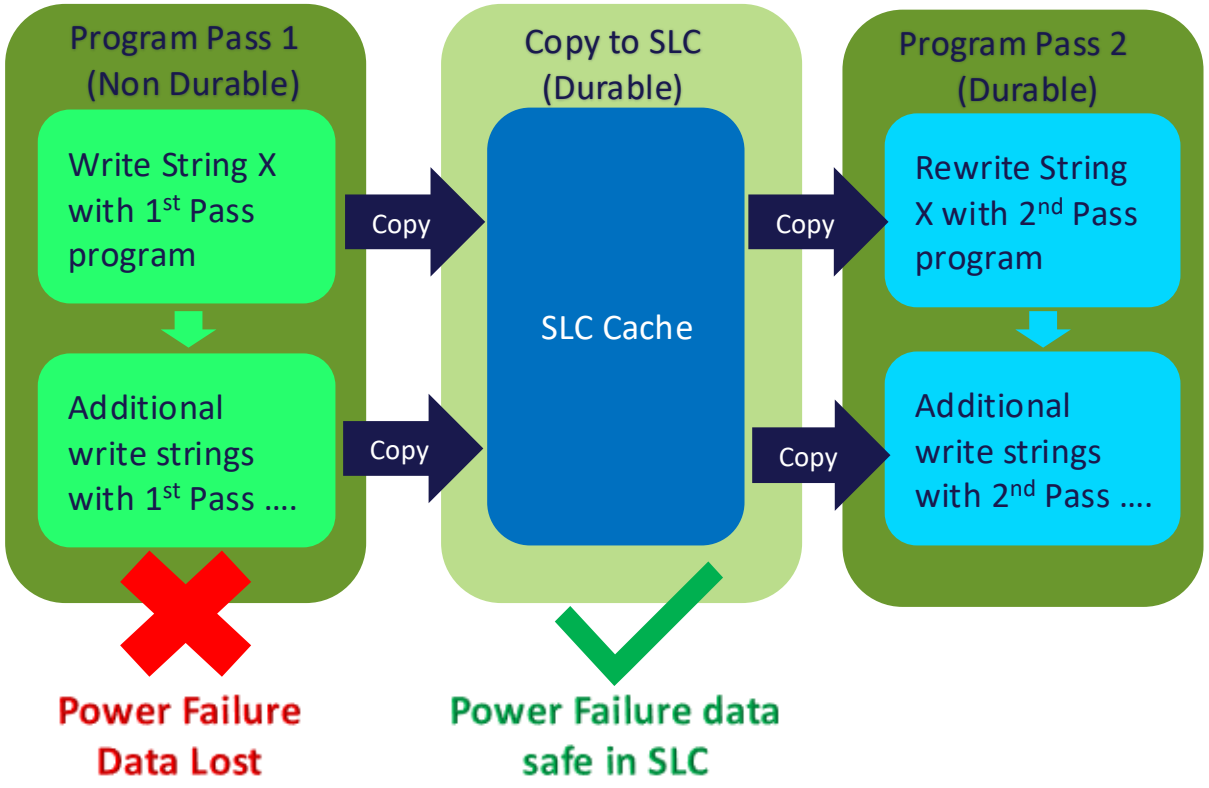
Holdup Capacitors **don't scale** with the increasing NAND performance

Lost Data on PFAIL



SLC Cache Provides a Solution

SLC Cache is a pool of pSLC blocks within the QLC SSD that are used as a host data temporary cache



Bonus: SLC Copy through controller can provide error correction

But SLC Cache has significant tradeoffs

SLC cache comes out of the SSD Overprovisioning

The SLC Cache size is based on the endurance ratio between QLC and pSLC

If SLC Cache sized incorrectly it can fail prior to the QLC regions



1:100 Endurance Ratio X
25% Bit Density
= 4% total drive capacity!

This will cause issues for drives with lower OP

Must be sized for worst case scenarios

Lack of host visibility into wear, bad blocks, etc.

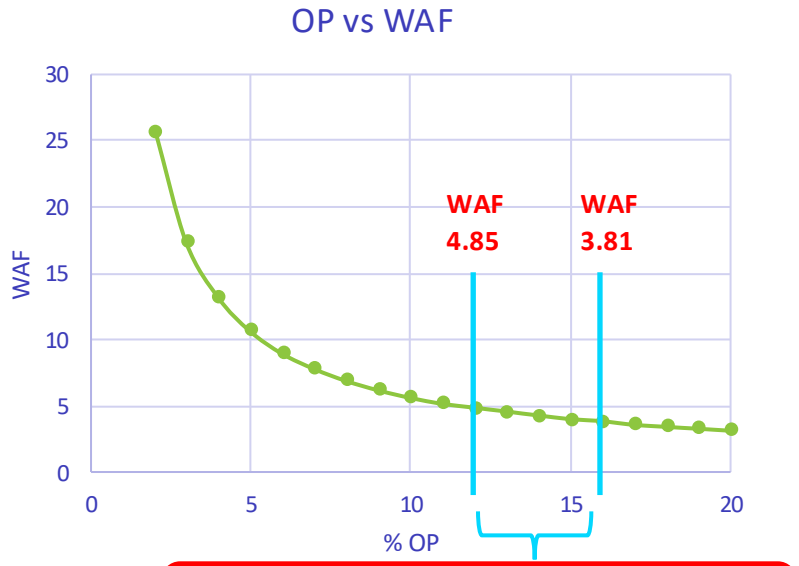
When SLC Cache eats some SSD OP, then what?

The SSD OP drives the **Write Amplification Factor** (WAF)

that equals the extra background writes the SSD does internally to free up invalid data capacity, this is often called Garbage Collection

This background Garbage Collection can generate about **4x times** the host random write bandwidth!

This means host random write **performance is reduced**



4% OP reduction causes 27% Increase in NAND writes!

WAF to OP curve using standard log-n formula

Impact of the extra SLC Reads and Writes

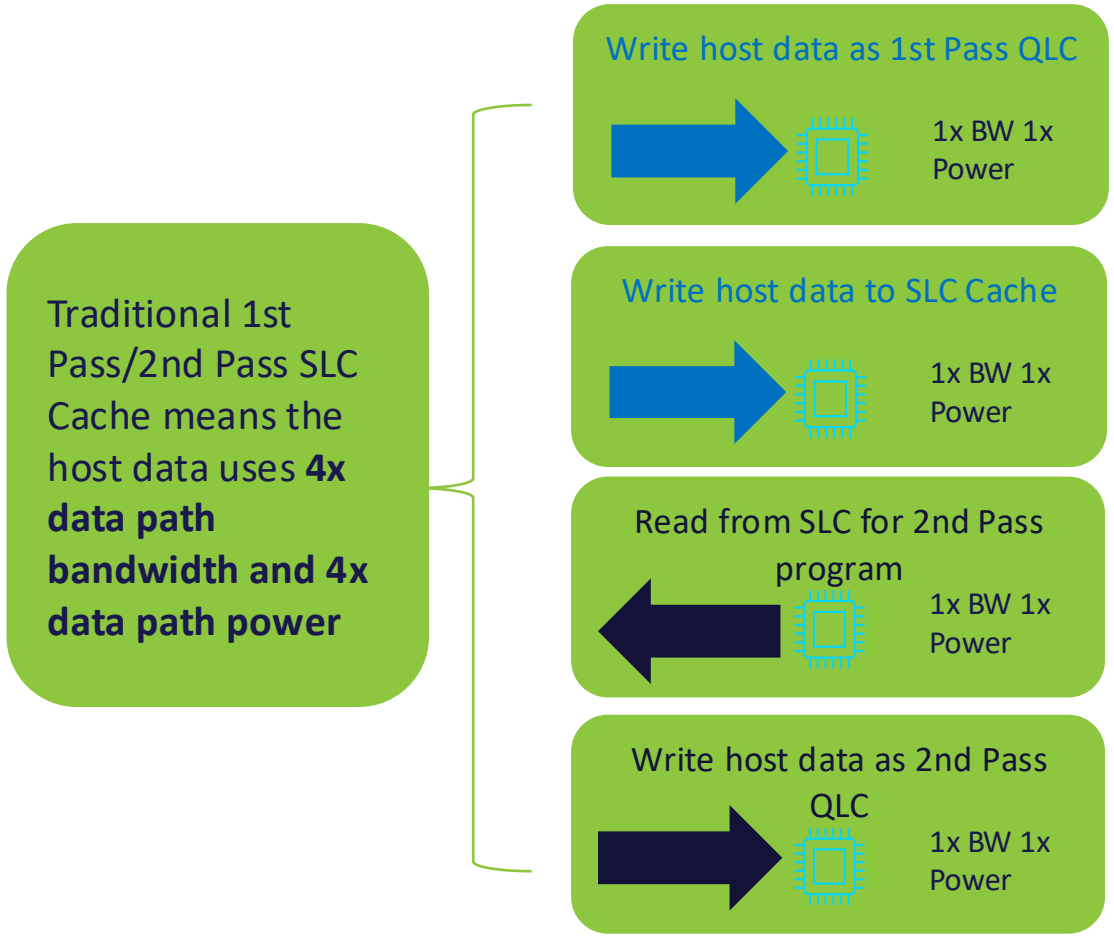
The extra SLC reads and writes also use up drive power!

Modelling SLC Cache power shows about **1W power penalty per 1GB/s** of host writes

But QLC writes keep **getting faster each generation**, random write performance is expected to **scale linearly with capacity**

And this power penalty is **on top of the WAF increase**

Traditional 1st Pass/2nd Pass SLC Cache means the host data uses **4x data path bandwidth and 4x data path power**



Using Direct Write QLC programming solves these issues…

- For QLC drives, POWER is becoming the primary factor in scaling QLC write performance
  - Anything that improves the write power efficiency for QLC writes will mean lower power drives for the required write performance, saving power at the Datacenter level
- **Direct Write** is our name for a new generation of QLC programming method where the 1st pass program is readable, which completely removes the need for caching the host write data in SLC
  - This eliminates WAF impact from the SLC Cache capacity (because there is no SLC Cache)
  - This eliminates power or bandwidth impact from copying the host data into SLC and out again
- *Call to action: As QLC adoption continues to increase across the industry, the industry should all aim to optimize QLC write with this type of technology*

Thank You!

